

STRUKTURALNI (STRUCTURAL) DESIGN PATTERNI – PRIMJENA NA DIJAGRAM KLASA (REZERVACIJE PUTOVANJA)

1. ADAPTER

Ideja: Pretvara interfejs jedne klase u interfejs koji klijentu odgovara, da bi različite komponente mogle raditi zajedno.

Primjena na naš dijagram:

- **Plaćanje:** različiti provajderi (kartica, banka, rate) imaju različite API-je. Adapter sloj “prevodi” pozive na zajednički interfejs
- **Notifikacije:** email/SMS/push servisi imaju različite metode slanja; adapter ujednačava pozive

Prednosti:

- Omogućava integraciju eksternih servisa bez mijenjanja domenskih klasa
- Smanjuje kompleksnost (manje if/else grananja)
- Poboljšava testabilnost

Mane:

- Više klasa i dodatni sloj apstrakcije
- Može postati kompleksan ako su API-ji veoma različiti

2. BRIDGE

Ideja: Razdvaja apstrakciju od implementacije, da se mogu mijenjati nezavisno.

Primjena na naš dijagram:

- **Notifikacija:** apstrakcija je “tip notifikacije” (potvrda rezervacije/plaćanja), a implementacija je “kanal” (email/SMS/push)
- **Plaćanje:** apstrakcija “plaćanje”, implementacije procesiranja (kartica/banka/rate)

Prednosti:

- Sprječava eksploziju broja klasa (kombinacije tipova i kanala)
- Lakše dodavanje novih kanala ili novih tipova poruka/plaćanja

Mane:

- Teže razumljivo na početku (dvije hijerarhije)
 - Može biti “overengineering” za mali sistem
-

3. COMPOSITE

Ideja: Omogućava da se pojedinačni objekti i grupe objekata tretiraju jednak.

Primjena na naš dijagram:

- **PlanPutovanja + StavkaPlana:** plan može sadržati dane (kompozit), a dan aktivnosti (list)
- **Paket + UslugaPaketa:** paket se može posmatrati kao kompozit od stavki/usluga, uz mogućnost grupisanja

Prednosti:

- Jednostavno agregiranje (ukupna cijena, ispis plana, validacija)
- Lako proširenje novim tipovima stavki

Mane:

- Teže nametnuti ograničenja
- Može sakriti razliku između “lista” i “kompozita”

4. DECORATOR

Ideja: Dinamički dodaje funkcionalnosti objektu bez mijenjanja njegove osnovne klase.

Primjena na naš dijagram:

- **Obračun cijene:** popust, takse, osiguranje, VIP usluge, dodatne usluge. Umjesto gomile logike u Rezervacija ili Paket, dekorateri dodaju step by step
- **Notifikacije:** dekorater za logging, retry, slanje kopije adminu, formatiranje template-a

Prednosti:

- Fleksibilno kombinovanje dodataka bez promjene osnovnog koda
- Manje “if/else” i bolja modularnost

Mane:

- Povećava broj malih klasa/objekata
 - Debugovanje može biti teže
-

5. FACADE

Ideja: Jednostavan interfejs prema kompleksnom podsistemu.

Primjena na naš dijagram:

- Proces **Rezervacije** obuhvata: izbor paketa, provjeru dostupnosti, kreiranje rezervacije, unos putnika, plaćanje, notifikaciju, generisanje plana
- RezervacijaFacade/BookingService može nuditi metodu tipa “kreirajRezervaciju(…)”

Prednosti:

- Jednostavniji API za UI/front-end ili kontrolere
- Smanjuje spregu prema internim klasama i olakšava održavanje

Mane:

- Fasada može postati prevelika
- Može sakriti napredne mogućnosti podsistema

6. FLYWEIGHT

Ideja: Dijeljenje zajedničkog stanja između velikog broja sličnih objekata radi uštede memorije.

Primjena na naš dijagram:

- **Destinacija:** više rezervacija referencira iste destinacije → dijeljenje/caching
- **Šifrnici/enumeracije:** Uloga, StatusRezervacije, MetodaPlaćanja, TipNotifikacije, StatusPaketa

Prednosti:

- Ušteda memorije i manje duplikata podataka
- Centralizovano upravljanje šifrnicima

Mane:

- Kompleksnost u keširanju i upravljanju instancama
 - Ne isplati se ako nema puno ponavljanja
-

7. PROXY

Ideja: Kontrolirše pristup objektu (lazy load, caching, security, logging).

Primjena na naš dijagram:

- **PlanPutovanja:** generisanje plana može biti skupo → proxy ga učita/generiše tek kad zatreba
- **Paket/Destinacija:** caching proxy prema bazi ili servisu
- **Notifikacije:** proxy može dodati retry/rate limit/log

Prednosti:

- Bolje performanse (lazy loading, cache)
- Centralno dodavanje sigurnosnih provjera i logovanja

Mane:

- Komplikovaniji tok izvršavanja (teže pratiti šta je stvarni objekat)
- Rizik razlika u ponašanju ako proxy ne imitira u potpunosti original

ZAKLJUČAK

- **Facade:** orkestracija procesa rezervacije (Rezervacija + Plaćanje + Notifikacija + PlanPutovanja)
- **Adapter:** integracije plaćanja i notifikacija
- **Decorator:** obračun cijena (popusti/takse/dodaci)
- **Composite:** plan putovanja (dan → aktivnosti) ili paket kao skup usluga
- **Proxy:** lazy generisanje plana i caching podataka
- **Bridge:** odvajanje tipa notifikacije od kanala slanja
- **Flyweight:** šifrnici i dijeljene destinacije/usluge

U našem sistemu smo primijenili **Facade** da proces rezervacije bude jednostavan za korištenje (jedan poziv pokreće provjeru paketa, kreiranje rezervacije, plaćanje i slanje notifikacije). Također smo primijenili **Decorator** da obračun cijene bude fleksibilan, tj. da lako dodajemo popuste i dodatne usluge bez mijenjanja postojeće logike.